

# Guía de Tecnologías, Arquitectura y Despliegue

Taller de Integración II y Taller de Integración IV

Segundo semestre 2026

## 1. Objetivo

El presente documento define las principales tecnologías, la arquitectura general, la estructura del monorepo y los procedimientos básicos de desarrollo y despliegue utilizados durante el trabajo conjunto de Taller de Integración II (TI2) y Taller de Integración IV (TI4).

Los objetivos de esta guía son:

- establecer un stack tecnológico común;
- explicar la responsabilidad de cada tecnología;
- justificar las principales decisiones tecnológicas;
- definir la estructura inicial del monorepo;
- explicar cómo ejecutar los proyectos localmente;
- definir cómo se comunican las aplicaciones con el backend;
- establecer cómo se gestionan variables de entorno y secretos;
- explicar cómo desplegar las diferentes partes del sistema;
- establecer claramente las responsabilidades técnicas de TI2 y TI4;
- reducir decisiones técnicas innecesarias durante el desarrollo.

Este documento complementa el protocolo de trabajo con Git, GitHub y Linear.

## 2. Principio general de arquitectura

El proyecto corresponde a un único producto y se organizará en **un monorepo GitHub**. El monorepo contendrá las aplicaciones Web y Mobile, el backend compartido y los paquetes internos que se incorporen durante el desarrollo.

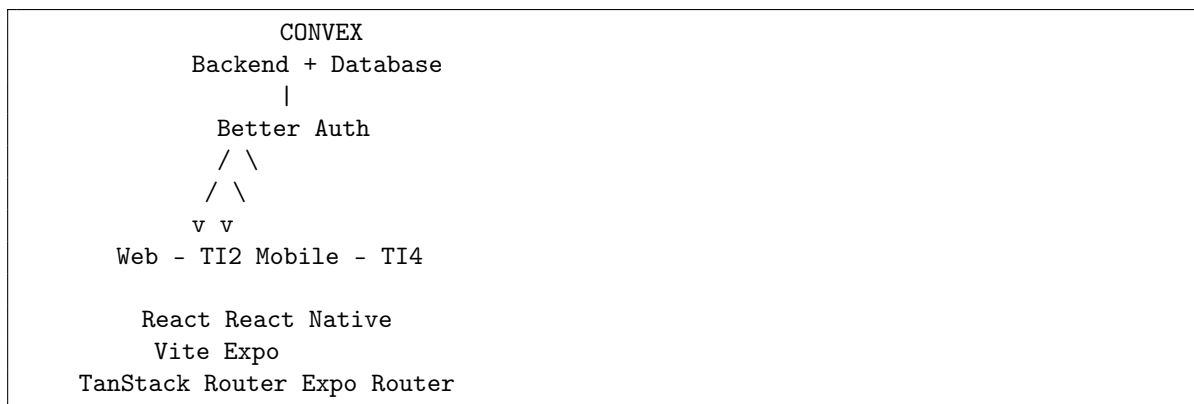
```
Proyecto
|
+-- apps/
| +-- web/ Aplicación Web de TI2
| +-- mobile/ Aplicación Mobile de TI4
|
+-- convex/ Backend, base de datos y autenticación
+-- packages/ Código compartido cuando corresponda
+-- .github/workflows/ Integración continua
+-- package.json Configuración de workspaces
+-- bun.lock Único archivo de bloqueo
```

Ambas aplicaciones utilizarán:

- el mismo backend;

- la misma base de datos;
- el mismo sistema de autenticación;
- las mismas reglas de negocio.

La arquitectura general será:



Por lo tanto, TI2 será propietario técnico del backend compartido y TI4 consumirá los servicios ofrecidos por TI2 desde la misma base de código.

### 3. Stack tecnológico

Las principales tecnologías seleccionadas son:

| Componente              | Tecnología                  |
|-------------------------|-----------------------------|
| Lenguaje principal      | TypeScript                  |
| Frontend Web            | React                       |
| Build Web               | Vite                        |
| Routing Web             | TanStack Router             |
| Aplicación móvil        | React Native + Expo         |
| Routing móvil           | Expo Router                 |
| Backend                 | Convex                      |
| Base de Datos           | Convex                      |
| Autenticación           | Better Auth                 |
| Build móvil             | EAS Build                   |
| Hosting Web             | Vercel                      |
| Hosting Backend         | Convex Cloud                |
| Control de versiones    | Git + GitHub                |
| Gestión del trabajo     | Linear                      |
| Integración continua    | GitHub Actions              |
| Organización del código | Monorepo con Bun workspaces |

Los comandos de esta guía utilizarán **bun** como gestor de paquetes y se ejecutarán desde la raíz del monorepo, salvo cuando se indique explícitamente una aplicación.

Se deberá mantener un único gestor de paquetes y un único archivo de bloqueo en la raíz del monorepo. Las aplicaciones y paquetes internos utilizarán los workspaces definidos en el `package.json` raíz.

### 3.1. Instalación de Bun

Antes de ejecutar los comandos de esta guía, cada integrante deberá instalar Bun en su equipo y verificar que esté disponible en la terminal.

En macOS y Linux se utilizará:

```
curl -fsSL https://bun.sh/install | bash
```

En Windows se utilizará PowerShell:

```
powershell -c "irm bun.sh/install.ps1|iex"
```

La mayoría de los integrantes de TI2 utilizará Windows, por lo que esa será la instalación de referencia para dicho equipo. Después de instalar Bun, se deberá abrir una nueva terminal y comprobar la instalación mediante:

```
bun --version
```

Por ejemplo, no deberán coexistir en el monorepo:

```
bun.lock
package-lock.json
pnpm-lock.yaml
yarn.lock
```

## 4. TypeScript

### 4.1. Responsabilidad

TypeScript será el lenguaje principal de programación de ambos equipos.

Será utilizado en:

- frontend web;
- aplicación móvil;
- funciones del backend;
- definición del modelo de datos;
- autenticación;
- contratos entre clientes y backend.

### 4.2. Justificación

Utilizar TypeScript en todo el proyecto permite compartir un mismo modelo mental entre ambos equipos y detectar una gran cantidad de errores durante el desarrollo antes de ejecutar la aplicación.

Además, Convex permite obtener tipos de extremo a extremo entre sus funciones de backend y los clientes TypeScript.

Como regla general, el código nuevo deberá escribirse en TypeScript.

## 5. React

React será utilizado para construir la interfaz de usuario de la aplicación web de TI2.

React será responsable de:

- componentes visuales;
- formularios;
- composición de pantallas;
- interacción con el usuario;
- presentación de los datos obtenidos desde Convex.

React no será responsable de acceder directamente a la base de datos.

El flujo esperado será:



## 6. Vite

Vite será utilizado como herramienta de desarrollo y construcción de la aplicación web.

Sus responsabilidades principales serán:

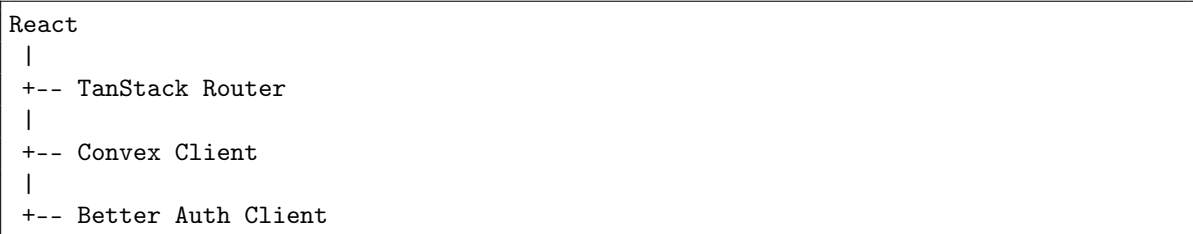
- ejecutar el servidor de desarrollo;
- procesar TypeScript y JSX;
- administrar módulos durante desarrollo;
- generar el build de producción.

### 6.1. Justificación

La aplicación web funcionará principalmente como cliente de Convex.

Por esta razón no es necesario introducir un segundo framework full-stack con una capa adicional de servidor.

La arquitectura será deliberadamente simple:



La lógica de servidor permanecerá en Convex.

## 7. TanStack Router

TanStack Router será utilizado como sistema de routing de la aplicación web.

Será responsable de:

- definición de páginas;
- navegación;
- parámetros de URL;
- layouts;
- rutas anidadas;
- navegación type-safe.

Se utilizará routing basado en archivos.

Por defecto las rutas se almacenarán en:

```
src/routes/
```

TanStack Router generará automáticamente:

```
src/routeTree.gen.ts
```

Este archivo es generado automáticamente y no deberá modificarse manualmente.

## 7.1. Justificación

Se utilizará TanStack Router en lugar de TanStack Start porque el servidor del proyecto ya está definido mediante Convex.

TanStack Start incorporaría funcionalidades adicionales de servidor que no son necesarias para la arquitectura definida.

## 8. Convex

Convex será la plataforma central del backend.

Será utilizado como:

- backend;
- base de datos;
- plataforma de ejecución de funciones;
- mecanismo de sincronización de datos con los clientes.

El backend será desarrollado y mantenido por TI2.

### 8.1. Tipos de funciones

La lógica principal se implementará mediante:

**Queries:** operaciones de lectura.

**Mutations:** operaciones que modifican datos.

**Actions:** operaciones que requieren interacción con servicios externos o comportamientos que no corresponden a una query o mutation.

Conceptualmente:

```
Cliente
|
+-- Query -----> Leer
|
+-- Mutation ----> Modificar
```

```
|  
+-- Action -----> Servicio externo / proceso especial
```

La mayor parte de la lógica de negocio deberá permanecer en queries y mutations cuando corresponda.

## 9. Base de datos Convex

El modelo de datos será administrado mediante Convex.

El esquema principal se encontrará en:

```
convex/schema.ts
```

El esquema deberá formar parte del control de versiones.

Los cambios relevantes al modelo de datos deberán:

- realizarse mediante una branch;
- pasar por Pull Request;
- considerar el impacto sobre la aplicación web;
- considerar el impacto sobre la aplicación móvil;
- quedar asociados al issue correspondiente en Linear.

No deberán existir cambios manuales indispensables que solamente conozca un integrante del equipo.

## 10. Better Auth

Better Auth administrará la autenticación de los usuarios.

La instancia principal de Better Auth será ejecutada sobre el backend Convex.

Entre sus responsabilidades se encuentran:

- registro de usuarios;
- inicio de sesión;
- cierre de sesión;
- manejo de sesiones;
- mecanismos adicionales de autenticación habilitados por el proyecto.

### 10.1. Autenticación y autorización

Es importante distinguir:

**Autenticación:** determina quién es el usuario.

**Autorización:** determina qué puede hacer el usuario.

Por lo tanto:

```
Better Auth  
|  
v  
¿Quién es el usuario?  
|  
v
```

```
Convex
|
v
¿Puede ejecutar esta operación?
```

Las reglas de autorización propias del negocio deberán implementarse en el backend y no confiar exclusivamente en que una pantalla esté oculta en el frontend.

## 11. Expo y React Native

TI4 desarrollará la aplicación móvil utilizando React Native mediante Expo.

Expo será responsable de proporcionar:

- entorno de desarrollo móvil;
- herramientas de ejecución;
- configuración de Android e iOS;
- integración con APIs nativas;
- proceso de build mediante EAS.

### 11.1. Justificación

Expo reduce la cantidad de configuración nativa que el equipo debe administrar directamente y permite mantener la mayor parte del desarrollo en TypeScript y React.

Esto permite que ambos equipos compartan tecnologías y conceptos.

| TI2 Web         | TI4 Mobile   |
|-----------------|--------------|
| React           | React Native |
| TanStack Router | Expo Router  |
| TypeScript      | TypeScript   |
| Convex          | Convex       |
| Better Auth     | Better Auth  |

## 12. Expo Router

Expo Router será utilizado para administrar la navegación de la aplicación móvil.

Las pantallas estarán representadas mediante archivos dentro de:

```
app/
```

Por ejemplo:

```
app/
|
+-- _layout.tsx
+-- index.tsx
+-- login.tsx
|
+-- usuarios/
    |
    +-- index.tsx
    +-- [id].tsx
```

Expo Router será responsable de:

- navegación;
- layouts;
- parámetros;
- deep links;
- organización de pantallas.

La lógica de negocio no deberá implementarse dentro del router.

## 13. Monorepo y un solo backend

El backend Convex existirá solamente en la carpeta `convex/` de la raíz del monorepo. TI4 **no** deberá crear una segunda carpeta Convex ni copiar funciones del backend.

La arquitectura correcta será:

```
Monorepo
|
+-- convex/
| |
| +-- schema.ts
| +-- users.ts
| +-- auth...
|
+-- apps/
|
+-- web/
+-- mobile/
```

De esta forma existe una única implementación del servidor y ambas aplicaciones consumen la misma API pública.

## 14. Tipos compartidos dentro del monorepo

Convex genera una representación TypeScript de su API pública. En el monorepo esa representación estará disponible para las aplicaciones Web y Mobile desde la misma base de código.

Conceptualmente:

```
convex/
|
| define funciones y genera tipos
v
convex/_generated/api
| |
v v
apps/web apps/mobile
```

Ambas aplicaciones utilizarán los tipos generados por Convex mediante un alias o un paquete interno definido por el monorepo. La configuración concreta del alias podrá variar, pero no deberá duplicar la implementación del backend ni mantener un contrato generado de forma independiente para cada aplicación.



La dependencia de Convex se declarará en los workspaces que corresponda y se instalará desde la raíz:

```
bun add convex --cwd apps/web
bun add convex --cwd apps/mobile
```

El comando de desarrollo del backend se ejecutará desde la raíz:

```
bunx convex dev
```

La carpeta `convex/_generated/` deberá mantenerse de acuerdo con la política del proyecto para archivos generados. Si la configuración de Convex permite regenerarla automáticamente en cada entorno, no se deberá crear una segunda copia manual dentro de `apps/mobile/`.

### 14.1. Regla

Web y Mobile consumen la API pública definida en `convex/`.

TI4 puede consumir las funciones del backend y utilizar sus tipos, pero no debe modificar directamente las funciones Convex que no estén dentro de su responsabilidad.

## 15. Estructura del monorepo

La estructura inicial será aproximadamente:

```
proyecto/
|
|-- apps/
| |
| | |-- web/
| | | |-- src/
| | | |-- public/
| | | |-- package.json
| | | |-- vite.config.ts
| | |
| | |-- mobile/
| | |-- app/
| | |-- components/
| | |-- features/
| | |-- package.json
| | |-- app.json
| |
|-- packages/
| |-- ui/ Componentes compartidos si corresponde
| |-- shared/ Tipos o utilidades compartidas si corresponde
|
|-- convex/
| |-- _generated/
| |-- betterAuth/
| |-- schema.ts
| |-- auth.config.ts
| |-- convex.config.ts
| |-- http.ts
| |-- ...
|
|-- .github/
|-- workflows/
```

```
|
+-- .env.example
+-- .gitignore
+-- package.json
+-- bun.lock
+-- tsconfig.json
+-- vercel.json
+-- README.md
```

Los directorios `packages/ui/` y `packages/shared/` son opcionales. Solo deberán crearse cuando exista una responsabilidad compartida real; no se incorporarán como capas vacías por anticipación.

El directorio del backend:

```
convex/_generated/
```

se generará desde la configuración de Convex y no se deberá modificar manualmente.

La aplicación móvil utilizará el backend y los paquetes compartidos desde este monorepo; no tendrá una copia local del backend ni un archivo de contrato externo.

## 16. Setup inicial del monorepo

Todos los comandos siguientes se ejecutarán desde la raíz del monorepo.

### 16.1. 1. Crear la raíz y los workspaces

La raíz deberá declarar los workspaces de las aplicaciones y de los paquetes internos:

```
{
  "name": "proyecto",
  "private": true,
  "workspaces": [
    "apps/*",
    "packages/*"
  ]
}
```

### 16.2. 2. Crear aplicación Web

```
bun create vite@latest apps/web -- --template react-ts
bun install
```

### 16.3. 3. Crear aplicación Mobile

```
bunx create-expo-app@latest apps/mobile
bun install
```

La plantilla estándar de Expo incorpora Expo Router.

### 16.4. 4. Instalar Convex en la raíz

El backend Convex se configurará en la raíz del monorepo:

```
bun add convex
bunx convex dev
```

La primera ejecución permitirá vincular o crear el proyecto Convex y generará la carpeta:

```
convex/
```

## 16.5. 5. Instalar TanStack Router en Web

```
bun add @tanstack/react-router --cwd apps/web
bun add -d @tanstack/router-plugin --cwd apps/web
bun add -d @tanstack/react-router-devtools --cwd apps/web
```

En Vite se deberá agregar el plugin de TanStack Router antes del plugin de React.

Ejemplo:

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import { tanstackRouter } from '@tanstack/router-plugin/vite'

export default defineConfig({
  plugins: [
    tanstackRouter({
      target: 'react',
      autoCodeSplitting: true,
    }),
    react(),
  ],
})
```

## 16.6. 6. Instalar Better Auth en el backend

```
bun add better-auth @convex-dev/better-auth
```

La integración deberá realizarse dentro del backend Convex siguiendo la estructura oficial de Better Auth para Convex.

Entre otros archivos, se utilizarán:

```
convex/
|
+-- auth.config.ts
+-- convex.config.ts
+-- http.ts
|
+-- betterAuth/
  |
  +-- convex.config.ts
  +-- auth.ts
  +-- schema.ts
  +-- adapter.ts
```

El esquema de Better Auth deberá generarse mediante sus herramientas y no mantenerse manualmente si puede evitarse.

## 17. Dependencias de la aplicación Mobile

TI4 solamente utilizará el cliente Convex y los tipos generados desde la carpeta compartida del backend. No deberá crear ni desplegar una segunda implementación.

### 17.1. 1. Instalar Better Auth para Expo

```
bun add better-auth @better-auth/expo --cwd apps/mobile
bun add expo-network expo-secure-store --cwd apps/mobile
```

Better Auth utilizará Secure Store para almacenar información de sesión de forma apropiada para la aplicación móvil.

Si el proyecto utiliza proveedores sociales, podrán requerirse además dependencias propias de Expo como linking o web browser.

## 18. Conexión de la aplicación Web a Convex

La aplicación web utilizará una instancia de:

```
ConvexReactClient
```

configurada mediante la URL del deployment.

La variable pública utilizada por Vite será:

```
VITE_CONVEX_URL=
```

Dentro del frontend:

```
const convex = new ConvexReactClient(
  import.meta.env.VITE_CONVEX_URL
)
```

El cliente deberá proveerse en la raíz de la aplicación.

## 19. Conexión de la aplicación Mobile a Convex

Expo utilizará el mismo cliente React de Convex.

La variable será:

```
EXPO_PUBLIC_CONVEX_URL=
```

En la raíz de Expo Router podrá configurarse:

```
const convex = new ConvexReactClient(
  process.env.EXPO_PUBLIC_CONVEX_URL!
)
```

y proveerse a la aplicación mediante el provider correspondiente.

Esto significa que:

```

Web
|
+----\
      \
      > mismo deployment Convex
      /
Mobile /

```

## 20. URLs de Convex

Convex utiliza URLs con responsabilidades distintas.

Conceptualmente tendremos:

**Convex URL:** conexión del cliente React con las funciones Convex.

**Convex Site URL:** endpoints HTTP, incluyendo rutas utilizadas por Better Auth cuando corresponda.

Por ejemplo, los clientes podrán necesitar variables equivalentes a:

```

VITE_CONVEX_URL=
VITE_CONVEX_SITE_URL=

```

para web, y:

```

EXPO_PUBLIC_CONVEX_URL=
EXPO_PUBLIC_CONVEX_SITE_URL=

```

para mobile.

Estas URLs permiten localizar servicios públicos.

**No son el lugar donde deben almacenarse secretos.**

## 21. Variables de entorno

Se distinguirán claramente variables públicas y secretos.

### 21.1. Frontend Web

Vite expone al código cliente variables con prefijo:

```
VITE_
```

Por ejemplo:

```
VITE_CONVEX_URL=
```

Toda variable `VITE_` debe considerarse visible para el usuario final.

### 21.2. Aplicación móvil

Expo expone al código cliente las variables con prefijo:

```
EXPO_PUBLIC_
```

Por ejemplo:

```
EXPO_PUBLIC_CONVEX_URL=
```

Toda variable `EXPO_PUBLIC_` debe considerarse pública.

### 21.3. Backend

Los secretos utilizados por las funciones Convex deberán configurarse en el entorno de Convex.

Por ejemplo:

```
BETTER_AUTH_SECRET
```

Estos valores no deberán estar:

- dentro del código;
- dentro del frontend;
- dentro de variables públicas;
- versionados en Git.

## 22. Archivos de entorno

Los archivos locales que contengan valores específicos de cada desarrollador no deberán subirse al monorepo.

Cada aplicación podrá mantener su propio archivo local dentro de su directorio:

```
apps/web/.env.local  
apps/mobile/.env.local
```

El monorepo deberá mantener ejemplos de configuración sin secretos:

```
apps/web/.env.example  
apps/mobile/.env.example
```

con las variables necesarias pero sin secretos.

Ejemplo TI2:

```
VITE_CONVEX_URL=  
VITE_CONVEX_SITE_URL=
```

Ejemplo TI4:

```
EXPO_PUBLIC_CONVEX_URL=  
EXPO_PUBLIC_CONVEX_SITE_URL=
```

## 23. Ejecución local del monorepo

Durante el desarrollo normalmente se utilizarán al menos dos procesos desde la raíz del monorepo.

### 23.1. Terminal 1: Convex

```
bunx convex dev
```

Este proceso sincroniza los cambios del backend con el deployment de desarrollo.

### 23.2. Terminal 2: Web

```
bun --cwd apps/web run dev
```

Conceptualmente:

```
Terminal 1 Terminal 2
bunx convex dev bun --cwd apps/web run dev
  | |
  v v
Backend Convex React + Vite
```

## 24. Ejecución local de Mobile

La aplicación móvil se ejecutará mediante Expo.

```
bun --cwd apps/mobile run start
```

o equivalentemente:

```
cd apps/mobile && bunx expo start
```

Desde esta sesión se podrá ejecutar la aplicación en:

- Android;
- iOS cuando el entorno lo permita;
- dispositivo físico;
- emulador o simulador.

La aplicación Mobile deberá conectarse al deployment de desarrollo de Convex definido para el proyecto.

No es necesario ejecutar el backend para modificar exclusivamente la aplicación Mobile, aunque sí deberá utilizarse un deployment válido para probar la integración.

## 25. Ambientes

Se distinguirán al menos:

**Development:** programación y pruebas durante el desarrollo.

**Production:** versión estable utilizada para las demostraciones y entregas.

En móvil también podrá existir un ambiente:

**Preview:** build destinado a pruebas del equipo.

Los entornos deberán utilizar configuraciones separadas cuando sea necesario.

## 26. Build de la aplicación Web

Antes de considerar una versión lista para integración deberá poder generarse el build de producción.

```
bun --cwd apps/web run build
```

Una aplicación que funciona solamente con:

```
bun --cwd apps/web run dev
```

pero cuyo build falla no se considera lista para integrar.

## 27. Despliegue del backend Convex

Durante desarrollo se utilizará:

```
bunx convex dev
```

Para desplegar funciones, esquema e índices hacia producción se utilizará:

```
bunx convex deploy
```

El deployment de producción será responsabilidad de TI2 y se ejecutará desde la raíz del monorepo.

No todos los integrantes deberán manipular manualmente producción.

## 28. Despliegue de la aplicación Web

La aplicación web será desplegada mediante Vercel.

Vercel se conectará al único repositorio GitHub y se configurará con **apps/web/** como directorio de la aplicación. Las órdenes de instalación y build deberán respetar el **bun.lock** ubicado en la raíz.

El flujo general será:

```
GitHub
|
v
main
|
v
Vercel
|
+---- Build apps/web
|
+---- Deploy Convex
|
v
Producción
```

Para integrar el despliegue de Convex con Vercel deberá configurarse un deploy key de producción como variable segura en Vercel:

```
CONVEX_DEPLOY_KEY
```



El build podrá ejecutar:

```
bunx convex deploy --cmd 'bun --cwd apps/web run build'
```

De esta manera se reduce el riesgo de publicar un frontend incompatible con las funciones Convex correspondientes.

## 29. Configuración SPA en Vercel

TanStack Router administrará las rutas en el cliente.

Por ello, al solicitar directamente una ruta como:

```
/usuarios/123
```

el hosting deberá entregar la aplicación y permitir que TanStack Router resuelva la ruta.

El repositorio podrá incluir:

```
vercel.json
```

con una regla equivalente a:

```
{
  "rewrites": [
    {
      "source": "/(.*)",
      "destination": "/index.html"
    }
  ]
}
```

## 30. Build de la aplicación móvil

Los builds para instalar serán generados mediante EAS Build. La configuración de EAS deberá utilizar `apps/mobile/` como directorio de la aplicación y compartir el lockfile de la raíz cuando el flujo de CI se ejecute desde el monorepo.

Primero deberá configurarse EAS para el proyecto.

Una configuración simplificada de:

```
eas.json
```

podrá incluir:

```
{
  "build": {
    "development": {
      "developmentClient": true
    },
    "preview": {
      "distribution": "internal"
    },
    "production": {}
  }
}
```

## 31. Build Preview de Mobile

Durante el curso no será necesario publicar cada versión en Google Play o App Store.

Para pruebas se utilizará preferentemente un build interno.

Android:

```
eas build --platform android --profile preview
```

iOS:

```
eas build --platform ios --profile preview
```

EAS proporcionará un build que podrá compartirse con integrantes y personas autorizadas para pruebas.

En Android, este mecanismo permite disponer de un APK para instalar sin depender de una publicación en Play Store.

## 32. Build de producción Mobile

Cuando corresponda generar una versión de producción:

```
eas build --platform android
```

o:

```
eas build --platform ios
```

También puede utilizarse:

```
eas build --platform all
```

La publicación efectiva en tiendas será una decisión independiente del proceso de desarrollo.

## 33. Flujos de despliegue

### 33.1. TI2

```
Monorepo GitHub
|
v
main
|
v
Vercel
|
+--> React/Vite
|
+--> Convex Production
|
v
Web desplegada
```

### 33.2. TI4

```
Monorepo GitHub
|
v
main
|
v
EAS Build
|
+--> Preview
|
+--> Production
|
v
Aplicación para instalar
```

## 34. Integración Continua

GitHub Actions verificará los cambios del monorepo antes de integrarlos.

La instalación deberá realizarse desde la raíz para utilizar el lockfile único:

```
bun install --frozen-lockfile
```

Cuando los scripts correspondientes existan, deberán ejecutarse como mínimo las verificaciones de los componentes afectados:

1. instalación de dependencias desde la raíz;
2. chequeo de TypeScript;
3. lint;
4. pruebas automatizadas disponibles;
5. build de Web, Mobile o backend según las rutas modificadas.

Conceptualmente:

```
Pull Request
|
v
bun install --frozen-lockfile
|
v
Type Check
|
v
Lint
|
v
Tests
|
v
Build
|
v
Code Review
|
v
```

El despliegue no reemplaza al proceso de CI.

## 35. Responsabilidades de TI2

TI2 será responsable técnicamente de:

- desarrollar la aplicación web;
- mantener React y TanStack Router;
- implementar las funciones Convex;
- definir y mantener el modelo de datos;
- implementar Better Auth;
- implementar reglas de autorización;
- desplegar el backend;
- desplegar la aplicación web;
- mantener disponibles los servicios requeridos por TI4;
- mantener estable la API pública consumida por Mobile;
- informar cambios incompatibles;
- mantener actualizados los tipos generados y los paquetes compartidos cuando correspon-da.

TI2 es propietario técnico del backend compartido.

## 36. Responsabilidades de TI4

TI4 será responsable técnicamente de:

- desarrollar la aplicación móvil mediante Expo;
- organizar la navegación mediante Expo Router;
- integrar la aplicación con Convex;
- integrar los flujos de Better Auth;
- consumir los tipos generados desde la configuración compartida;
- detectar y reportar problemas de integración;
- generar builds mediante EAS;
- evitar duplicar lógica perteneciente al backend.

Además, TI4 mantiene las responsabilidades de gestión, coordinación, seguimiento y control de calidad definidas en el documento metodológico general.

## 37. Frontera entre TI2 y TI4

La frontera técnica entre ambos equipos estará formada principalmente por las funciones públicas de Convex, el sistema de autenticación compartido y los paquetes internos que sean consumidos por más de una aplicación.

TI4 no deberá:

- crear una segunda base de datos;
- crear una segunda implementación del backend;
- copiar las funciones Convex dentro de `apps/mobile/`;
- modificar directamente producción de TI2;

- duplicar reglas de negocio únicamente para hacer funcionar Mobile.

TI2 deberá mantener suficientemente documentadas las funciones públicas consumidas por Mobile.

Como mínimo deberá conocerse:

- propósito;
- argumentos;
- resultado;
- requerimientos de autenticación;
- errores relevantes.

## 38. Cambios que afectan a ambos equipos

Un cambio deberá coordinarse mediante Linear cuando modifique:

- nombre de una función pública;
- argumentos de una función;
- tipo de retorno;
- reglas de autenticación;
- reglas de autorización;
- estructura de datos utilizada por ambos clientes;
- comportamiento funcional esperado por la aplicación Mobile;
- un paquete compartido utilizado por Web y Mobile.

Estas dependencias deberán registrarse en Linear.

Por ejemplo:

```
TI2-42 Cambiar flujo de autenticación
|
+-- bloquea -->
                    TI4-18
                    Adaptar login mobile
```

## 39. Qué corresponde a cada capa

### React

Interfaz de usuario Web.

No contiene acceso directo a la base de datos.

### TanStack Router

Navegación Web.

No contiene lógica de negocio.

### Expo / React Native

Interfaz de usuario Mobile.

No reemplaza el backend.

## Expo Router

Navegación Mobile.

No contiene reglas de negocio compartidas.

## Convex

Backend, persistencia y reglas de negocio compartidas.

## Better Auth

Identidad y sesiones.

No reemplaza las reglas de autorización del sistema.

## 40. Ejemplo de distribución de responsabilidades

Supongamos que debe implementarse:

*“Mostrar el perfil de un usuario autenticado.”*

La responsabilidad se distribuirá así:

```

Better Auth
  |
  +--> identifica al usuario actual
  |
  v
Convex
  |
  +--> valida permisos
  +--> obtiene información
  |
  +-----+
  | |
  v v
Web Mobile
React React Native
```

El frontend decide **cómo mostrar** la información.

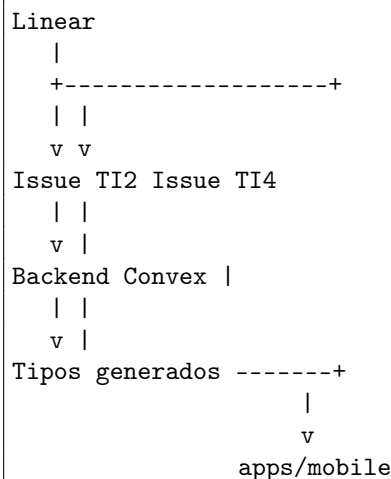
El backend decide **qué información puede obtener** el usuario.

## 41. Proceso para una funcionalidad compartida

Una funcionalidad que afecte a ambos equipos podrá seguir el siguiente flujo:

1. TI4 realiza el triage y se identifican las tareas necesarias.
2. TI2 implementa o modifica las funciones necesarias en Convex.
3. TI2 y TI4 integran el cambio mediante uno o más Pull Requests relacionados.
4. se regeneran o actualizan los tipos públicos de Convex;
5. TI4 implementa el consumo desde la aplicación Mobile;
6. ambos equipos prueban la integración;
7. los cambios llegan a los procesos de despliegue de Web y Mobile.

Conceptualmente:



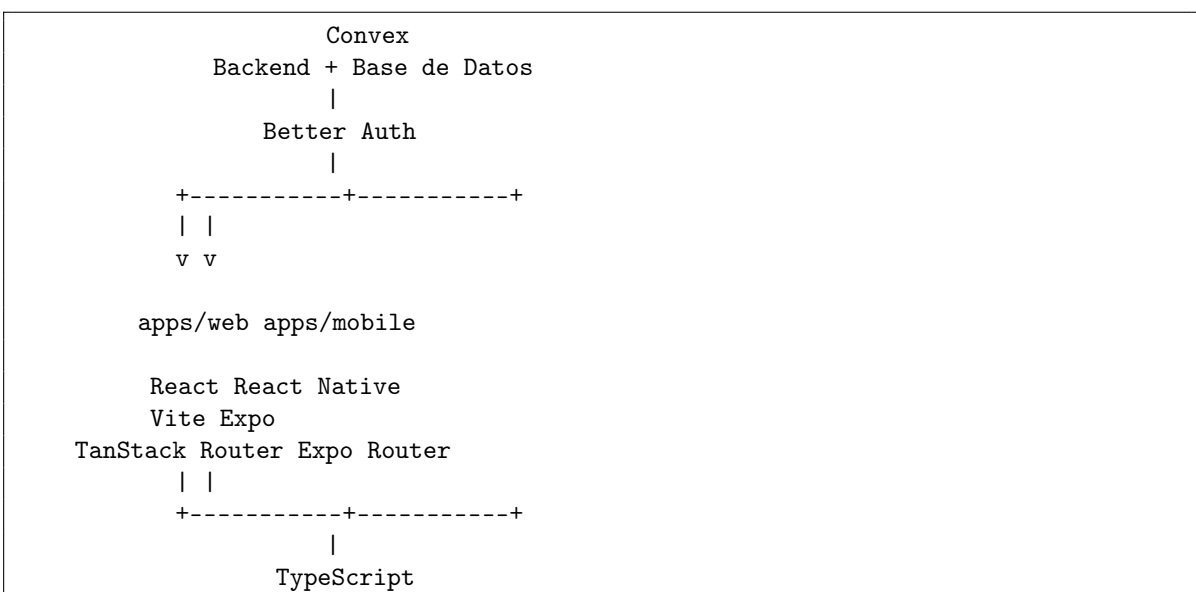
## 42. Principios técnicos

Durante el desarrollo se aplicarán los siguientes principios:

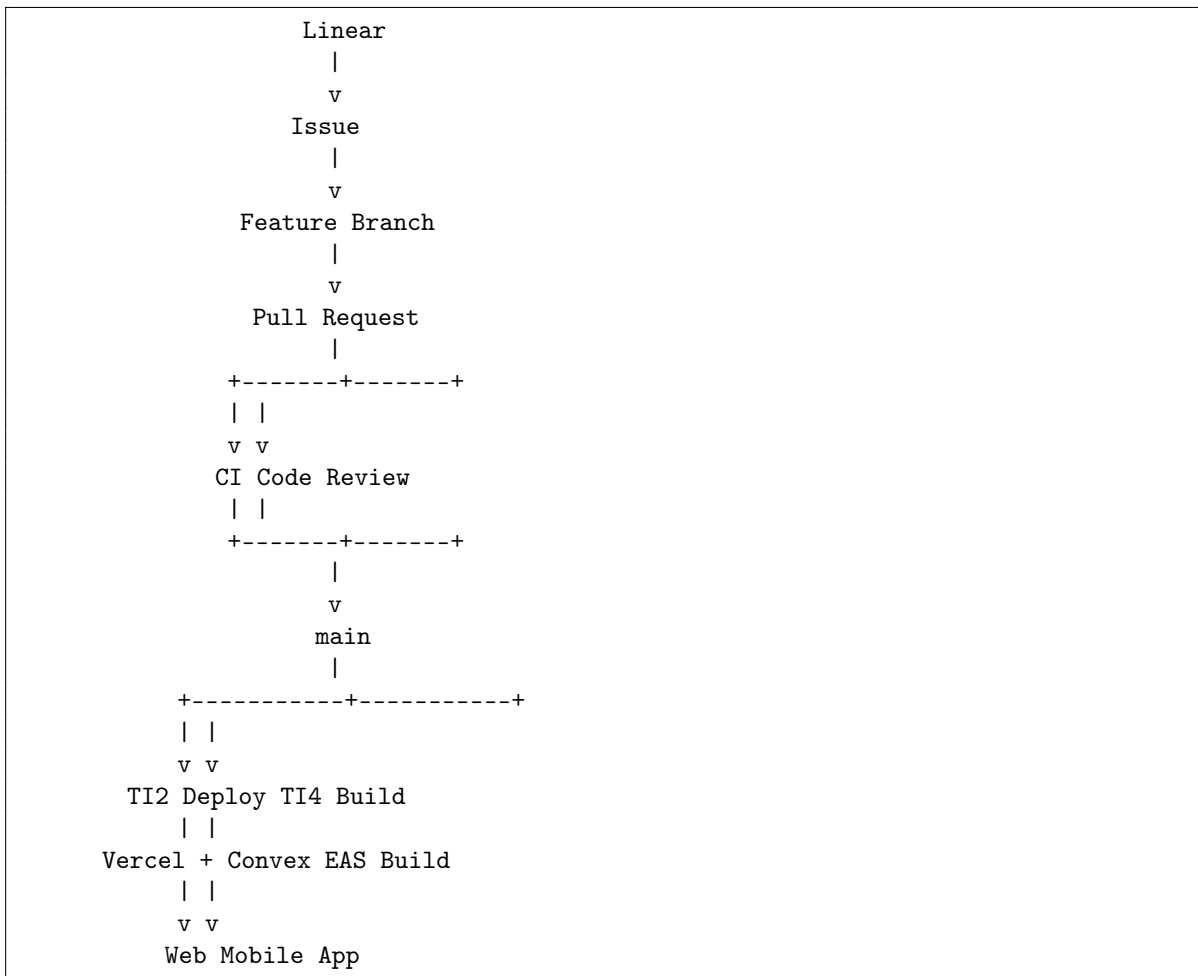
1. existe un solo backend;
2. existe una sola fuente de verdad para los datos;
3. Web y Mobile consumen los mismos servicios;
4. los secretos nunca se almacenan en código cliente;
5. las reglas de autorización se validan en el backend;
6. la API pública y los paquetes compartidos deben ser explícitos;
7. los cambios incompatibles deben coordinarse antes de integrarse;
8. el código generado necesario para compilar debe mantenerse correctamente versionado;
9. una aplicación que no puede generar su build no está terminada;
10. los entornos de desarrollo y producción deben mantenerse separados.

## 43. Resumen de arquitectura

El stack tecnológico puede resumirse de la siguiente manera:



El flujo de desarrollo y despliegue será:



## 44. Referencias oficiales

Las configuraciones específicas pueden evolucionar con nuevas versiones de las herramientas. Antes de realizar cambios importantes deberán consultarse sus documentaciones oficiales.

- TanStack Router: <https://tanstack.com/router/latest>
- TanStack Router con Vite: [Documentación de TanStack Router con Vite](#)
- Convex: <https://docs.convex.dev/>
- Convex con React: <https://docs.convex.dev/client/react/overview>
- Convex con React Native: <https://docs.convex.dev/quickstart/react-native>
- Convex en Vercel: <https://docs.convex.dev/production/hosting/vercel>
- Better Auth: <https://better-auth.com/>
- Better Auth con Convex: <https://better-auth.com/docs/integrations/convex>
- Better Auth con Expo: <https://better-auth.com/docs/integrations/expo>
- Expo: <https://docs.expo.dev/>
- EAS Build: <https://docs.expo.dev/build/introduction/>
- EAS Internal Distribution: <https://docs.expo.dev/build/internal-distribution/>